1. Gramática usada para el analizador sintáctico

```
CHUNK ::= BLOCK

BLOCK ::= STAT_LIST

STAT_LIST ::= STAT STAT_LIST
           | epsilon

STAT ::= ID_NAME PRIMARY_STAT_TAIL
       | local VAR_OR_FUNC_DECL
       | global VAR_OR_FUNC_DECL
       | function FUNC_NAME FUNC_BODY
       | while EXP do BLOCK end
       | repeat BLOCK until EXP
       | if EXP then BLOCK ELSEIF_LIST ELSE_OPT end
       | for ID_NAME FOR_TAIL
       | goto ID_NAME
       | tkn_goto ID_NAME tkn_goto
       | break
       # Segun el test Campuswire 6, se debe tener return aqui
       | return RET_TAIL
       | tkn_semicolon

ID_NAME ::= id
          | print
          | warn
          | pcall
          | xpcall
          | error

ELSEIF_LIST ::= elseif EXP then BLOCK ELSEIF_LIST
              | epsilon

ELSE_OPT ::= else BLOCK
          | epsilon

RET_TAIL ::= epsilon
          | VAL_EXP_LIST

VAR_OR_FUNC_DECL ::= function ID_NAME FUNC_BODY
                   | NAME_DECL_LIST OPT_ASSIGN

NAME_DECL_LIST ::= NAME_DECL NAME_DECL_LIST_TAIL

NAME_DECL_LIST_TAIL ::= tkn_comma NAME_DECL NAME_DECL_LIST_TAIL
                      | epsilon

NAME_DECL ::= ID_NAME NAME_DECL_TAIL

NAME_DECL_TAIL ::= tkn_opening_bra EXP tkn_closing_bra
                 | epsilon

OPT_ASSIGN ::= tkn_assign VAL_EXP_LIST
             | epsilon

PRIMARY_STAT_TAIL ::= tkn_assign VAL_EXP_LIST
                    | tkn_comma VAR_LIST tkn_assign VAL_EXP_LIST
                    | tkn_colon ID_NAME CALL_TAIL
                    | tkn_opening_par ARGS_OPT tkn_closing_par
     CALL_STAT_TAIL
                    | tkn_period ID_NAME PRIMARY_STAT_TAIL
                    | tkn_opening_bra EXP tkn_closing_bra
     PRIMARY_STAT_TAIL

CALL_TAIL ::= tkn_opening_par ARGS_OPT tkn_closing_par CALL_STAT_TAIL

CALL_STAT_TAIL ::= tkn_opening_par ARGS_OPT tkn_closing_par
     CALL_STAT_TAIL
                 | tkn_colon ID_NAME CALL_TAIL
                 | tkn_period ID_NAME PRIMARY_STAT_TAIL
                 | tkn_opening_bra EXP tkn_closing_bra PRIMARY_STAT_TAIL
                 | epsilon

VAR_LIST ::= VAR_ITEM VAR_LIST_TAIL

VAR_LIST_TAIL ::= tkn_comma VAR_ITEM VAR_LIST_TAIL
                | epsilon

VAR_ITEM ::= ID_NAME VAR_SUFFIX

VAR_SUFFIX ::= tkn_period ID_NAME VAR_SUFFIX
             | tkn_opening_bra EXP tkn_closing_bra VAR_SUFFIX
             | epsilon

FOR_TAIL ::= tkn_assign EXP tkn_comma EXP FOR_STEP_OPT do BLOCK end
           | tkn_comma ID_NAME FOR_NAME_LIST in EXP_LIST do BLOCK end
           | in EXP_LIST do BLOCK end

FOR_STEP_OPT ::= tkn_comma EXP
               | epsilon

FOR_NAME_LIST ::= tkn_comma ID_NAME FOR_NAME_LIST
                | epsilon

FUNC_NAME ::= ID_NAME FUNC_NAME_TAIL

FUNC_NAME_TAIL ::= tkn_period ID_NAME FUNC_NAME_TAIL
                 | tkn_colon ID_NAME
                 | epsilon

FUNC_BODY ::= tkn_opening_par PAR_LIST_OPT tkn_closing_par BLOCK end

PAR_LIST_OPT ::= PAR_LIST
               | epsilon

PAR_LIST ::= id PAR_LIST_TAIL
           | tkn_varargs

PAR_LIST_TAIL ::= tkn_comma PAR_ITEM
                | epsilon

PAR_ITEM ::= id PAR_LIST_TAIL
           | tkn_varargs

VAL_EXP ::= function FUNC_BODY
          | EXP

VAL_EXP_LIST ::= VAL_EXP VAL_EXP_LIST_TAIL

VAL_EXP_LIST_TAIL ::= tkn_comma VAL_EXP VAL_EXP_LIST_TAIL
                    | epsilon

EXP_LIST ::= EXP EXP_LIST_TAIL

EXP_LIST_TAIL ::= tkn_comma EXP EXP_LIST_TAIL
                | epsilon

ARGS_OPT ::= VAL_EXP_LIST
           | epsilon

EXP ::= AND_EXP EXP_TAIL

EXP_TAIL ::= or AND_EXP EXP_TAIL
           | epsilon

AND_EXP ::= REL_EXP AND_EXP_TAIL

AND_EXP_TAIL ::= and REL_EXP AND_EXP_TAIL
               | epsilon

REL_EXP ::= BIT_OR_EXP REL_EXP_TAIL

REL_EXP_TAIL ::= REL_OP BIT_OR_EXP
               | epsilon

REL_OP ::= tkn_less
         | tkn_greater
         | tkn_leq
         | tkn_geq
         | tkn_neq
         | tkn_equal

BIT_OR_EXP ::= BIT_XOR_EXP BIT_OR_EXP_TAIL

BIT_OR_EXP_TAIL ::= tkn_bit_or BIT_XOR_EXP BIT_OR_EXP_TAIL
                  | epsilon

BIT_XOR_EXP ::= BIT_AND_EXP BIT_XOR_EXP_TAIL

BIT_XOR_EXP_TAIL ::= tkn_bitex_or BIT_AND_EXP BIT_XOR_EXP_TAIL
                   | epsilon

BIT_AND_EXP ::= SHIFT_EXP BIT_AND_EXP_TAIL
```

```
BIT_AND_EXP_TAIL ::= tkn_bit_and SHIFT_EXP BIT_AND_EXP_TAIL
                   | epsilon

SHIFT_EXP ::= CONCAT_EXP SHIFT_EXP_TAIL

SHIFT_EXP_TAIL ::= tkn_left_shift CONCAT_EXP SHIFT_EXP_TAIL
                 | tkn_right_shift CONCAT_EXP SHIFT_EXP_TAIL
                 | epsilon

CONCAT_EXP ::= ADD_EXP CONCAT_EXP_TAIL

CONCAT_EXP_TAIL ::= tkn_concat CONCAT_EXP
                  | epsilon

ADD_EXP ::= MUL_EXP ADD_EXP_TAIL

ADD_EXP_TAIL ::= tkn_plus MUL_EXP ADD_EXP_TAIL
               | tkn_minus MUL_EXP ADD_EXP_TAIL
               | epsilon

MUL_EXP ::= UNARY_EXP MUL_EXP_TAIL

MUL_EXP_TAIL ::= tkn_times UNARY_EXP MUL_EXP_TAIL
               | tkn_div UNARY_EXP MUL_EXP_TAIL
               | tkn_floor_div UNARY_EXP MUL_EXP_TAIL
               | tkn_mod UNARY_EXP MUL_EXP_TAIL
               | epsilon

UNARY_EXP ::= not UNARY_EXP
            | tkn_length UNARY_EXP
            | tkn_minus UNARY_EXP
            | POW_EXP

POW_EXP ::= ATOM POW_EXP_TAIL

POW_EXP_TAIL ::= tkn_power UNARY_EXP
               | epsilon

ATOM ::= ID_NAME SUFFIX_EXP
       | valor_numerico
       | cadena_de_caracteres
       | nil
       | true
       | false
       | TABLE_CONSTRUCTOR
       | tkn_opening_par EXP tkn_closing_par

SUFFIX_EXP ::= tkn_period ID_NAME SUFFIX_EXP
             | tkn_opening_bra EXP tkn_closing_bra SUFFIX_EXP
             | tkn_opening_par ARGS_OPT tkn_closing_par SUFFIX_EXP
             | tkn_colon ID_NAME tkn_opening_par ARGS_OPT
      tkn_closing_par SUFFIX_EXP
             | epsilon

TABLE_CONSTRUCTOR ::= tkn_opening_key FIELD_LIST_OPT tkn_closing_key
```

```
FIELD_LIST_OPT ::= FIELD FIELD_LIST_TAIL
                 | epsilon

FIELD_LIST_TAIL ::= FIELD_SEP FIELD_OPT
                  | epsilon

FIELD_OPT ::= FIELD FIELD_LIST_TAIL
            | epsilon

FIELD_SEP ::= tkn_comma
            | tkn_semicolon

FIELD ::= tkn_opening_bra EXP tkn_closing_bra tkn_assign VAL_EXP
        | ID_NAME FIELD_ID_TAIL
        | function FUNC_BODY
        | EXP_NON_ID

FIELD_ID_TAIL ::= tkn_assign VAL_EXP
                | SUFFIX_EXP POW_EXP_TAIL MUL_EXP_TAIL ADD_EXP_TAIL
      SHIFT_EXP_TAIL BIT_AND_EXP_TAIL BIT_XOR_EXP_TAIL BIT_OR_EXP_TAIL
      REL_EXP_TAIL AND_EXP_TAIL EXP_TAIL

EXP_NON_ID ::= AND_EXP_NON_ID EXP_TAIL

AND_EXP_NON_ID ::= REL_EXP_NON_ID AND_EXP_TAIL

REL_EXP_NON_ID ::= BIT_OR_EXP_NON_ID REL_EXP_TAIL

BIT_OR_EXP_NON_ID ::= BIT_XOR_EXP_NON_ID BIT_OR_EXP_TAIL

BIT_XOR_EXP_NON_ID ::= BIT_AND_EXP_NON_ID BIT_XOR_EXP_TAIL

BIT_AND_EXP_NON_ID ::= SHIFT_EXP_NON_ID BIT_AND_EXP_TAIL

SHIFT_EXP_NON_ID ::= CONCAT_EXP_NON_ID SHIFT_EXP_TAIL

CONCAT_EXP_NON_ID ::= ADD_EXP_NON_ID CONCAT_EXP_TAIL

ADD_EXP_NON_ID ::= MUL_EXP_NON_ID ADD_EXP_TAIL

MUL_EXP_NON_ID ::= UNARY_EXP_NON_ID MUL_EXP_TAIL

UNARY_EXP_NON_ID ::= not UNARY_EXP
                   | tkn_length UNARY_EXP
                   | tkn_minus UNARY_EXP
                   | POW_EXP_NON_ID

POW_EXP_NON_ID ::= ATOM_NON_ID POW_EXP_TAIL

ATOM_NON_ID ::= valor_numerico
              | cadena_de_caracteres
              | nil
              | true
              | false
              | TABLE_CONSTRUCTOR
              | tkn_opening_par EXP tkn_closing_par
```

2. Gramática BNF base usada para la creación de la gramática

2.1. Gramática BNF Oficial Extraída de Lua 5.5 Reference Manual

A continuación se presentan todas las producciones sintácticas extraídas del manual de referencia oficial:

```
chunk ::= block

block ::= {stat}

stat ::= ';'
       | varlist '=' explist
       | functioncall
       | label
       | break
       | goto Name
       | do block end
       | while exp do block end
       | repeat block until exp
       | if exp then block {elseif exp then block} [else block] end
       | for Name '=' exp ',' exp [',' exp] do block end
       | for namelist in explist do block end
       | function funcname funcbody
       | local function Name funcbody
       | global function Name funcbody
       | local attnamelist ['=' explist]
       | global attnamelist ['=' explist]
       | global [attrib] '*'
       | return [explist] [';']

label ::= '::' Name '::'

funcname ::= Name {'.' Name} [':' Name]

varlist ::= var {',' var}

var ::= Name
      | prefixexp '[' exp ']'
      | prefixexp '.' Name

namelist ::= Name {',' Name}

explist ::= exp {',' exp}

exp ::= nil | false | true | Numeral | LiteralString | '...' |
     functiondef
      | prefixexp | tableconstructor | exp binop exp | unop exp

prefixexp ::= var | functioncall | '(' exp ')'
```

```
functioncall ::= prefixexp args
               | prefixexp ':' Name args

args ::= '(' [explist] ')'
       | tableconstructor
       | LiteralString

functiondef ::= function funcbody

funcbody ::= '(' [parlist] ')' block end

parlist ::= namelist [',' varargparam] | varargparam
```

```
varargparam ::= '...' [Name]

tableconstructor ::= '{' [fieldlist] '}'

fieldlist ::= field {fieldsep field} [fieldsep]

field ::= '[' exp ']' '=' exp | Name '=' exp | exp

fieldsep ::= ',' | ';'

attnamelist ::= [attrib] Name [attrib] {',' Name [attrib]}

attrib ::= '<' Name '>'
```

2.2. Cambios Realizados a la Gramática Original y Justificación Técnica

Para adaptar la gramática formal de Lua 5.5 a un analizador sintáctico predictivo descendente **LL(1)** y cumplir con los requerimientos de UNcode, se implementaron los siguientes cambios:

1. **Eliminación de recursión por la izquierda y estratificación de expresiones (Diseño LL(1)):**
   - *Original:* La regla `exp ::= exp binop exp | unop exp` y la recursión mutua entre `prefixexp`, `var` y `functioncall` poseen recursión por la izquierda directa e indirecta, además de ambigüedad en precedencia y asociatividad.
   - *Cambio:* Se descompuso `exp` en 12 niveles de precedencia jerárquica: `EXP` (`or`), `AND_EXP` (`and`), `REL_EXP` (relacionales), `BIT_OR_EXP` (`|`), `BIT_XOR_EXP` (`~`), `BIT_AND_EXP` (`&`), `SHIFT_EXP` (`<<`, `>>`), `CONCAT_EXP` (`..`), `ADD_EXP` (`+`, `-`), `MUL_EXP` (`*`, `/`, `//`, `%`), `UNARY_EXP` (`not`, `#`, `-`) y `POW_EXP` (`^`).
   - *Asociatividad:* Para operadores asociativos a derecha (`..` y `^`), la recursión gramatical se mantiene a la derecha; para asociatividad a izquierda, se introdujeron no terminales auxiliares de cola (`*_TAIL`) con derivación a `epsilon`.
2. **Factorización del conflicto en sentencias: Asignación vs Llamada a Función:**
   - *Original:* `stat ::= varlist '=' explist | functioncall`. Ambas sentencias inician mediante `prefixexp` (típicamente un identificador `Name`), lo que genera un conflicto First/First irresoluble con un solo token de anticipación.
   - *Cambio:* Se unificaron bajo la regla factorizada `STAT ::= ID_NAME PRIMARY_STAT_TAIL`. En `PRIMARY_STAT_TAIL` se procesan de forma uniforme accesos a campos (`.`), índices (`[...]`), llamadas (`(...)`) y métodos (`:`), bifurcando deterministamente entre llamada consumida o sentencia de asignación (`tkn_assign` o continuación de lista con `tkn_comma`).
3. **Exclusividad del token `do` para estructuras de ciclo:**
   - *Original:* `stat ::= do block end`.
   - *Cambio:* Se eliminó la sentencia de bloque explícito independiente. El token `do` queda reservado única y exclusivamente como delimitador en los ciclos `while ... do` y `for ... do`. Por tanto, `do` no pertenece a First(`STAT`) ni es esperado al inicio de una sentencia.
4. **Restricción de funciones anónimas en ciclo `for` numérico:**
   - *Original:* `stat ::= for Name '=' exp ',' exp [',' exp] do block end`.
   - *Cambio:* En la expresión inicial tras el signo `=` en la cabecera del ciclo numérico, se restringe la posibilidad de definir closures anónimos (`function funcbody`). Por ello se evalúa directamente con `EXP` (cuyo First no incluye `function`). En asignaciones estándar o retornos sí se admite función anónima mediante `VAL_EXP ::= function FUNC_BODY | EXP`.
5. **Unificación y simplificación de declaraciones `local` y `global`:**
   - *Original:* `stat ::= local attnamelist ['=' explist] | global attnamelist ['=' explist] | global [attrib] '*'`, soportando atributos como `<const>` o `<close>`.
   - *Cambio:* Se eliminaron los atributos y el comodín `global *`, unificando ambas sentencias bajo `VAR_OR_FUNC_DECL`. Tras `local` o `global` únicamente se permite una definición de función (`function ID_NAME FUNC_BODY`) o una lista de nombres de variables con inicialización opcional (`NAME_DECL_LIST OPT_ASSIGN`).
6. **Resolución de ambigüedad en constructores de tabla:**
   - *Original:* `field ::= '[' exp ']' '=' exp | Name '=' exp | exp`.
   - *Cambio:* Cuando un campo inicia con un identificador, existe un conflicto First/First entre la asignación `Name = exp` y una expresión que comience por dicho nombre (ej. `x + 1` o `f(x)`). Se resolvió factorizando mediante `FIELD ::= ID_NAME FIELD_ID_TAIL`, distinguiendo de inmediato si continúa una asignación (`tkn_assign`) o la evaluación de una expresión general (`SUFFIX_EXP ...`). Para expresiones que inician con otros términos se utiliza `EXP_NON_ID`.
7. **Validación estricta de parámetros en funciones y varargs:**
   - *Original:* `parlist ::= namelist [',' varargparam] | varargparam`, con `varargparam ::= '...' [Name]`.
   - *Cambio:* Se eliminó el nombre opcional en varargs (`... [Name]` → `tkn_varargs`). Además, en `PAR_LIST`, toda coma debe ir obligatoriamente seguida de un nuevo parámetro o `tkn_varargs`, no admitiéndose comas flotantes finales.
8. **Reutilización de funciones nativas como identificadores:**
   - *Original:* El manual utiliza únicamente el no terminal abstracto `Name`.
   - *Cambio:* El analizador léxico reconoce nombres de funciones del sistema (`print`, `warn`, `pcall`, `xpcall` y `error`) como

tokens con nombre propio. Como en Lua es válido redefinirlos o utilizarlos como variables y nombres de funciones propias, se definió la producción:

```
ID_NAME ::= id | print | warn | pcall | xpcall | error
```

permitiendo su uso en cualquier posición sintáctica donde se espere un identificador.

9. **Adaptación estricta a los tokens del Analizador Léxico y operaciones Bitwise binarias:**
   - *Cambio:* Se mapearon literales y operadores a los tokens exactos del analizador léxico: literales (`valor_numerico`, `cadena_de_caracteres`, `nil`, `true`, `false`) y operadores con prefijo `tkn_*` (`tkn_assign`, `tkn_semicolon`, `tkn_plus`, etc.).
   - *Operaciones bitwise:* Conforme a las instrucciones del proyecto, las operaciones a nivel de bits se definen exclusivamente como binarias entre dos operandos (`tkn_bit_and`, `tkn_bit_or`, `tkn_bitex_or`, `tkn_left_shift`, `tkn_right_shift`).
10. **Sentencia `return` permisiva dentro de bloques:**
   - *Original:* En Lua estándar, `return` se define típicamente como la última sentencia del bloque (`laststat`).
   - *Cambio:* Se integró `return RET_TAIL` como una alternativa regular dentro de `STAT`, permitiendo retornos en cualquier punto de la secuencia de sentencias (`STAT_LIST`) de acuerdo con las pruebas del curso.